Les nombres complexes

- Introduction aux complexes [AE18; Appendix I]
- Introduction aux polynômes [AE18; P.6]

[AE18] R. A. Adams and C. Essex. *Calculus : A Complete Course*. 9 Edition (Pearson, 2018).

Un nombre complexe est défini comme

$$z = a + bi, \quad i^2 = -1, \quad a, b \in \mathbb{R}.$$

On appelle a la partie réelle de z et b la partie complexe.

Module et argument. Le module d'un nombre complexe $z = a + ib$ est la distance à l'origine :

$$|z| = \sqrt{a^2 + b^2}.$$

L'argument de z, noté $\arg(z)$, est l'angle (en radians) entre le vecteur (a, b) et l'axe réel positif :

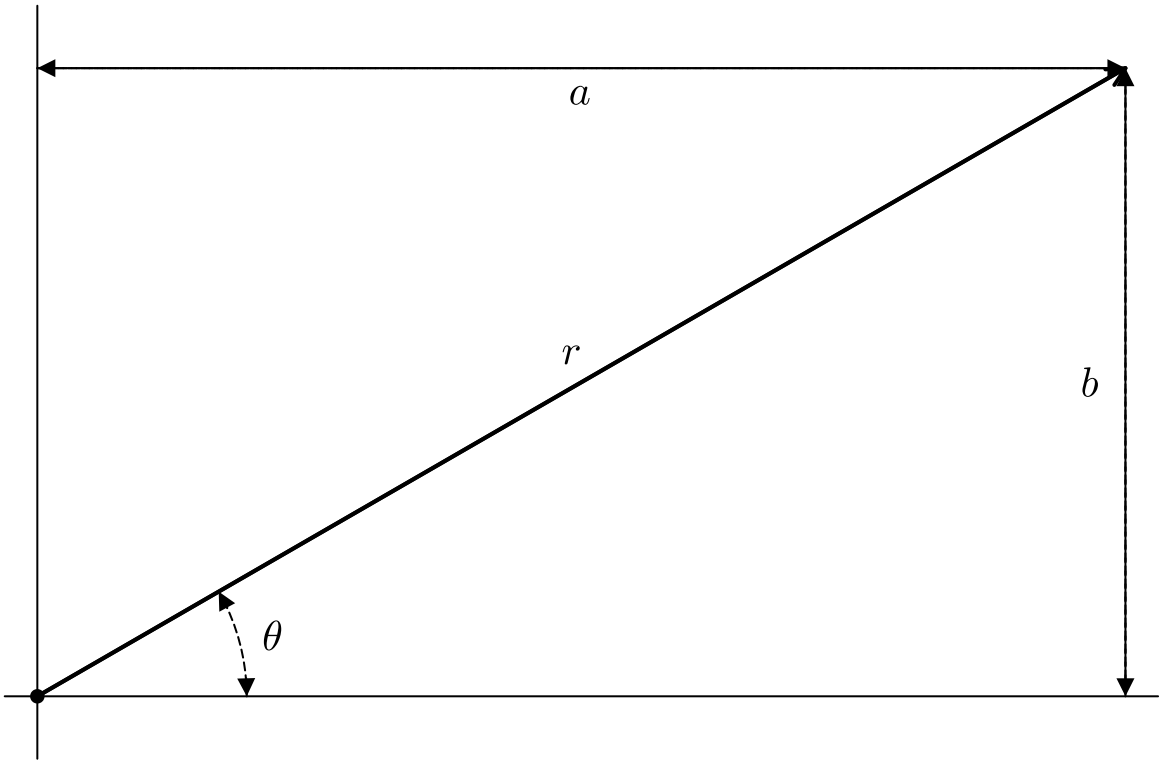
$$\arg(z) = \theta = \arctan\left(\frac{b}{a}\right).$$

Il est défini à $2k\pi$ près. L'argument principal est noté $\operatorname{Arg}(z) \in (-\pi, \pi]$.

Ceci est la représentation cartésienne de z, il existe aussi la représentation polaire

$$z = r(\cos(\theta) + i\sin(\theta)), \quad r \geq 0.$$

Cartésienne	Polaire	Exponentielle
$z = a + bi$	$z = r\operatorname{cis}(\theta)$	$z = r\exp(i\theta) = re^{i\theta}$
$a = r\cos(\theta)$	$r = \sqrt{a^2 + b^2}$	
$b = r\sin(\theta)$	$\theta = \tan^{-1}(b/a)$	

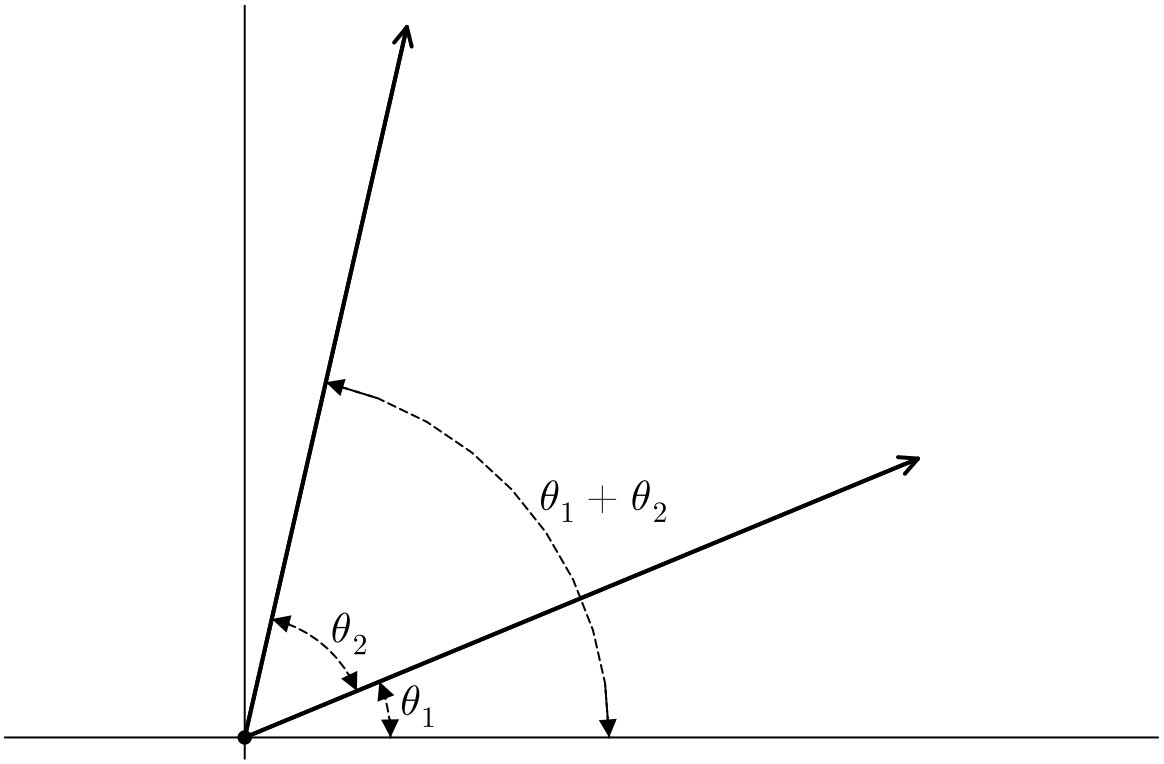


Représentation Cartésienne : facile pour addition, soustraction

$$(a_1 + b_1 i) \pm (a_2 + b_2 i) = (a_1 \pm a_2) + (b_1 \pm b_2)i$$

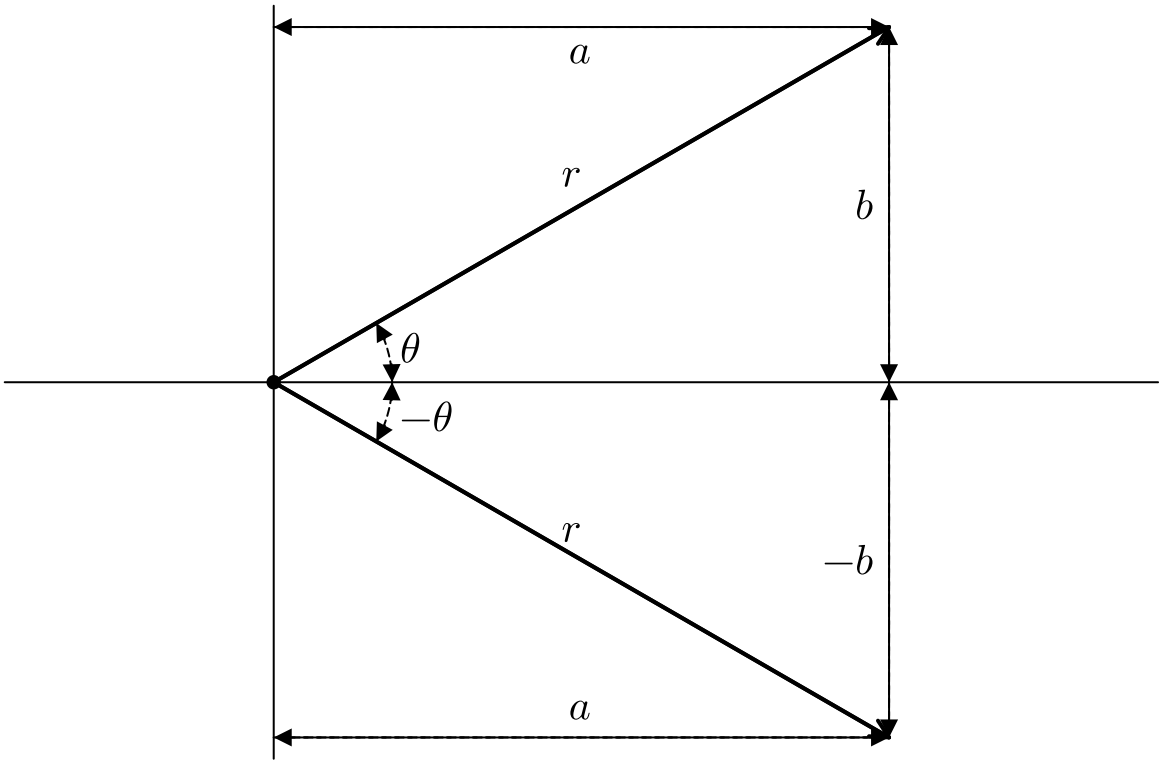
Représentation Polaire : facile pour multiplier, diviser. Géométriquement, cela correspond à une **rotation** d'angle θ

$$\begin{aligned} r_1 e^{i\theta_1} r_2 e^{i\theta_2} &= r_1 r_2 e^{i(\theta_1 + \theta_2)} \\ \frac{r_1 e^{i\theta_1}}{r_2 e^{i\theta_2}} &= \frac{r_1}{r_2} e^{i(\theta_1 - \theta_2)} \\ (re^{i\theta})^{-1} &= r^{-1} e^{-i\theta} \end{aligned}$$



Définition : *Complexe conjugué*

$$z = a + bi \qquad \bar{z} = a - bi$$
$$z = re^{i\theta} \qquad \bar{z} = re^{-i\theta}$$



Quiz : Quel est la valeur $z\bar{z}$? (vous pouvez choisir entre cartésienne ou polaire)

▶ **Réponse**

Résoudre des équations polynomiales

Les nombres complexes nous ont été imposés par des équations *réelles*

Considérons l'équation cubique réelle

$$x^3 = 15x + 4,$$

qui a pour solution réelle $x = 4$ (vérification : $4^3 = 64 = 15 \cdot 4 + 4$).

La formule de Cardan du 16e siècle pour les racines d'une cubique donne

$$x = \sqrt[3]{2 + \sqrt{-121}} \; + \; \sqrt[3]{2 - \sqrt{-121}}\,.$$

Cette formule nécessite de prendre la racine carrée d'un nombre *négatif*, alors même que l'équation, et sa solution $x = 4$, sont entièrement réelles.

Voyons comment le calcul se déroule. On écrit d'abord $\sqrt{-121} = 11i$:

$$x = \sqrt[3]{2 + 11i} \; + \; \sqrt[3]{2 - 11i}\,.$$

On peut vérifier que $2 + 11i$ et $2 - 11i$ sont eux-mêmes des cubes parfaits :

$$(2 + i)^3 = 8 + 12i + 6i^2 + i^3 = 8 + 12i - 6 - i = 2 + 11i,$$

et de même, par conjugaison, $(2 - i)^3 = 2 - 11i$. La formule de Cardan devient alors

$$x = \sqrt[3]{(2+i)^3} + \sqrt[3]{(2-i)^3} = (2 + i) + (2 - i) = 4.$$

On voit explicitement les parties imaginaires $+i$ et $-i$ apparaître dans le calcul intermédiaire, puis s'annuler dans la dernière étape pour redonner la solution réelle $x = 4$.

Morale. Les nombres complexes n'ont pas été introduits comme une généralisation abstraite pour elle-même. Ils se sont imposés en mathématiques parce qu'ils sont nécessaires comme étape intermédiaire pour calculer des réponses réelles parfaitement ordinaires. Les parties imaginaires apparaissent, font leur travail, puis s'annulent — un schéma que l'on retrouvera plus tard dans ce cours (par exemple dans la transformée de Fourier de signaux à valeurs réelles).

Le théorème fondamental de l'algèbre

Tout polynome $a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n$ de degré n a n racines complexes (ce qui n'exclut pas que certaines d'entre elles aient une partie imaginaire nulle et donc soient réelles). Dans cette liste de n racines, certaines apparaissent plusieurs fois. Mais qu'est-ce que ça signifie qu'une racine soit racine plusieurs fois ?

Multiplicité

Une valeur z est une racine du polynôme $P(x)$ si $P(z) = 0$. Si z est également une racine de la dérivée, c'est à dire $P'(z) = 0$ alors z est une racine double. En général, z est une racine de multiplicité m si $P(z) = P'(z) = \cdots = P^{(m)}(z) = 0$ où $P^{(k)}$ est la dérivée $m$ième de P.

De façon équivalente, ça revient à l'exposant du facteur $(x - z)$ dans la factorisation du polynôme. Par exemple, si :

$$P(x) = (x - z)^m \cdot Q(x),$$

où $Q(z) \neq 0$, alors z est une racine de multiplicité m.

Exemple

$$P(x) = (x - 1)^2 (x - 2)$$

Propriété des paires conjuguées

Si $z = a + ib$ est une racine de multiplicité m, alors la racine conjuguée $\overline{z} = a - ib$ possède également une multiplicité m.

▶ **Preuve**

Le cas quadratique

$$ax^2 + bx + c = 0 \qquad \Rightarrow \quad x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

▶ **Preuve**

Les racines de 1

Exemple introductif : les racines cubiques de 1

Quelles sont les solutions de $x^3 = 1$. En d'autre termes, quels sont les racines de $x^3 - 1$.

```
cube = -1 + x^3
```

```
cube = Polynomial([-1, 0, 0, 1])
```

On sait deviner une première racine : 1 car $1^3 = 1$.

```
première_racine = -1 + x
```

```
première_racine = Polynomial([-1, 1])
```

Grace à Horner, on sait factoriser cette racine, ce qui donne

$$x^3 - 1 = (x - 1)(x^2 + x + 1)$$

```
q = 1.0 + 1.0·x + 1.0·x^2
```

```
q = div(cube, première_racine)
```

```
-1.0 + 1.0·x^3
```

```
première_racine * q
```

On trouve alors les deux racines manquantes à l'aide des formules du cas quadratique

```
Δ = -3
```

```
Δ = 1 - 4
```

```
▸ (-0.5+0.866025im, -0.5-0.866025im)
```

```
(-1 + √(abs(Δ)) * im) / 2, (-1 - √(abs(Δ)) * im) / 2
```

Le cas général

Pour calculer les racines d'un polynôme de haut degré, il n'y a pas de formule. Mais on peut les calculer en reformulant le problème en problème de calcul de valeurs propre de la matrice companion. Comme on sait calculer numériquement les valeurs propres d'une matrices, ça nous permet de calculer numériquement les racines d'un polynôme.

Des valeurs propres aux racines d'un polynômes

Une matrice A a comme valeur propre λ avec vecteur propre associé x (non-nul) si

$$Ax = \lambda x$$

ou de façon équivalente

$$(A - \lambda I)x = 0$$

Si la matrice $A - \lambda I$ était régulière, il n'y aurait qu'une seule solution : $x = 0$. Seulement, le fait que λ soit une valeur propre est équivalente au fait qu'il existe une solution x non-nulle. Ceci équivaut à dire que le déterminant:

$$\det(A - \lambda I) = 0$$

Ce déterminant est un polynôme en λ de degré n.

Des racines d'un polynômes aux valeurs propres

Comment trouver les `n` racines ? Commençons par un exemple:

```
▸ (-1 + x, -2 + x, -3 + x)
1 p1, p2, p3 = Polynomial([-1, 1]), Polynomial([-2, 1]), Polynomial([-3, 1])
```

```
p = -6 + 11·x - 6·x² + x³
1 p = p1 * p2 * p3
```

```
▸ [1.0, 2.0, 3.0]
1 roots(p)
```

Comment est-ce que `roots` a retrouvé les racines à partir des coefficients ?

Commençons par essayer de trouver une des racines. Comment trouver r ?

$$(x - r)q = \alpha p$$

Où α est une constante et le polynôme q est de la forme

$$q = b_2x^2 + b_1x + b_0$$

En comparant les termes en x^3, on trouve que $\alpha = b_2$. On a donc:

$$xq - b_2p = rq$$
$$b_1x^2 + b_0x - b_2(-6x^2 + 11x - 6) = r(b_2x^2 + b_1x + b_0)$$

En comparant les coefficients des monômes 1, x et x^2, on a 3 équations:

$$\begin{bmatrix} 0 & 0 & 6 \\ 1 & 0 & -11 \\ 0 & 1 & 6 \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \end{bmatrix} = r \begin{bmatrix} b_0 \\ b_1 \\ b_2 \end{bmatrix}$$

4 inconnues pour seulement 3 équations, comment va-t-on trouver b_0, b_1, b_2 et r ?

▶ **Réponse**

```
E = Eigen{Float64, Float64, Matrix{Float64}, Vector{Float64}}
values:
3-element Vector{Float64}:
 0.9999999999999978
 2.000000000000007
 2.9999999999999925
vectors:
3×3 Matrix{Float64}:
  0.762001  -0.588348   0.534522
 -0.635001   0.784465  -0.801784
  0.127     -0.196116   0.267261
```

```
E = eigen([
    0 0 6
    1 0 -11
    0 1 6
])
```

```
0.9999999999999978
```

```
E.values[1]
```

```
2.000000000000007
```

```
E.values[2]
```

```
2.9999999999999925
```

```
E.values[3]
```

On trouve la première racine

```
(1.0, [0.762001, -0.635001, 0.127])
```

```
r, b = E.values[1], E.vectors[:, 1]
```

En normalisant, on trouve bien $(x-2)(x-3)$

```
[6.0, -5.0, 1.0]
```

```
b / b[3]
```

$6 - 5 \cdot x + x^2$

```
p2 * p3
```

Utilitaires

```
using Polynomials, LinearAlgebra, PlutoUI, Plots
```

```
import DocumenterCitations
```

```
qa (generic function with 2 methods)
```

```
include("utils.jl")
```

```
biblio =
```

▸ CitationBibliography("/home/runner/work/LSINC1113/LSINC1113/Lectures/biblio.bib", AlphaSt

```
biblio = load_biblio!()
```

```
ⓘ Loading bibliography from `/home/runner/work/LSINC1113/LSINC1113/Lectures/biblio.bib`...
```

```
⊗ Entry west2022Introduction is missing the publisher field(s).
```

```
ⓘ Loading completed.
```

```
cite (generic function with 1 method)
```

```
refs (generic function with 1 method)
```

```
refs(keys) = bibrefs(biblio, keys)
```

```
draw_angle (generic function with 1 method)
```

```
function draw_angle(r, from, to, label)
    θs = range(from, stop=to, length = 10)
    plot!(r * cos.(θs), r * sin.(θs), label = nothing, linestyle = :dash, color = :black, arrow = Plots.arrow(:both, :closed))
    _r = r + 0.02
    θ_mid = (from + to) / 2
    annotate!(_r * cos(θ_mid), _r * sin(θ_mid), text("\$" * label * "\$", :left))
end
```

```
draw_complex! (generic function with 1 method)
```

```
function draw_complex!(r, θ; cartesian = true, a_pos = :top, b_label = "b", θ_label = "\\theta", start_θ = 0, θ_r = 1/6)
    a = r * cos(θ)
    b = r * sin(θ)
    plot!([0, a], [0, b], arrow = true, label = nothing, linewidth=2, color = :black)
    if cartesian
        arrow = Plots.arrow(:both, :closed)
        plot!([a, a], [0, b], label = nothing, linestyle = :dash, color = :black, arrow = arrow)
        plot!([0, a], [b, b], label = nothing, linestyle = :dash, color = :black, arrow = arrow)
        annotate!(a/2, b, text("\$a\$", a_pos))
        annotate!(a - 0.02, b/2, text("\$" * b_label * "\$", :right))
        annotate!(a/2, b/2, text("\$r\$", :bottom, :right))
    end
    draw_angle(r * θ_r, 0, θ, θ_label)
end
```

```
draw_complex (generic function with 1 method)
```

```
function draw_complex(args...; kws...)
    scatter([0], [0], ratio = :equal, color = :black, label = nothing, framestyle = :origin, ticks = nothing)
    draw_complex!(args...; kws...)
end
```